

ME 7751 Homework 2

Toby Viet Nguyen

February 18, 2026

Scripts Included

`poisson_solver.py`, `plot_csv.py`, `homework2c.sh`, `homework2d.sh`, `homework2ei.sh`, `homework2eii.sh`, `homework2fg.sh`

Usage Instructions

All Bash scripts call functionality to `poisson_solver.py` and `plot_csv.py`. Run any script to regenerate all the images used for the problem. Run `python poisson_solver.py -h` to see available command-line arguments.

Problem 1a

Begin with the 2-D Poisson's equation with $\lambda = 1$.

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} + Q = 0$$

Discretize the domain into N^2 nodes with grid spacing $\Delta x = \Delta y = h = \frac{1}{N-1}$ such that...

$$x_0 = 0, x_1 = h, x_2 = 2h, \dots, x_i = ih, \dots, x_{N-1} = 1$$

$$y_0 = 0, y_1 = h, y_2 = 2h, \dots, y_j = jh, \dots, y_{N-1} = 1$$

Use central differences for both second derivatives.

$$\frac{T_{i-1,j} - 2T_{i,j} + T_{i+1,j}}{\Delta x^2} + \frac{T_{i,j-1} - 2T_{i,j} + T_{i,j+1}}{\Delta y^2} + Q_{i,j} = 0$$

$$\frac{1}{h^2}T_{i-1,j} + \frac{1}{h^2}T_{i+1,j} + \frac{1}{h^2}T_{i,j-1} + \frac{1}{h^2}T_{i,j+1} - \frac{4}{h^2}T_{i,j} = -Q_{i,j}$$

$$a_{i,j}^W T_{i-1,j} + a_{i,j}^E T_{i+1,j} + a_{i,j}^S T_{i,j-1} + a_{i,j}^N T_{i,j+1} + a_{i,j}^P T_{i,j} = -Q_{i,j}$$

Organize $T_{i,j}$ and $Q_{i,j}$ in row-major order, defining $\{T\}, \{Q\} \in \mathbb{R}^{N^2}$ as follows...

$$\{T\} = \begin{bmatrix} T_{0,0} & \dots & T_{N-1,0} & | & \dots & | & T_{0,N-1} & \dots & T_{N-1,N-1} \end{bmatrix}^\top$$

$$\{Q\} = \begin{bmatrix} -Q_{0,0} & \dots & -Q_{N-1,0} & | & \dots & | & -Q_{0,N-1} & \dots & -Q_{N-1,N-1} \end{bmatrix}^\top$$

Define $[A] : \mathbb{R}^{N^2} \mapsto \mathbb{R}^{N^2}$ with submatrices $[M_j], [L_j], [R_j] : \mathbb{R}^N \mapsto \mathbb{R}^N$ as...

$$[A] = \begin{bmatrix} [M_0] & [R_0] & & & & & \mathbf{0} \\ [L_1] & [M_1] & [R_1] & & & & \\ & [L_2] & [M_2] & [R_2] & & & \\ & & \ddots & \ddots & \ddots & & \\ & & & [L_{N-2}] & [M_{N-2}] & [R_{N-2}] \\ \mathbf{0} & & & & [L_{N-1}] & [M_{N-1}] \end{bmatrix}$$

$$[M_j] = \begin{bmatrix} a_{0,j}^P & a_{0,j}^E & & & & & \mathbf{0} \\ a_{1,j}^W & a_{1,j}^P & a_{1,j}^E & & & & \\ & a_{2,j}^W & a_{2,j}^P & a_{2,j}^E & & & \\ & & \ddots & \ddots & \ddots & & \\ & & & a_{N-2,j}^W & a_{N-2,j}^P & a_{N-2,j}^E \\ \mathbf{0} & & & & a_{N-1,j}^W & a_{N-1,j}^P \end{bmatrix}$$

$$[L_j] = \begin{bmatrix} a_{0,j}^N & & & & & & \mathbf{0} \\ & a_{1,j}^N & & & & & \\ & & a_{2,j}^N & & & & \\ & & & \ddots & & & \\ & & & & a_{N-2,j}^N & & \\ \mathbf{0} & & & & & a_{N-1,j}^N \end{bmatrix}$$

$$[R_j] = \begin{bmatrix} a_{0,j}^S & & & & & & \mathbf{0} \\ & a_{1,j}^S & & & & & \\ & & a_{2,j}^S & & & & \\ & & & \ddots & & & \\ & & & & a_{N-2,j}^S & & \\ \mathbf{0} & & & & & a_{N-1,j}^S \end{bmatrix}$$

To account for Dirichlet B.C. $T_{0,j} = 2y_j^3 - 3y_j^2 + 1, T_{N-1,j} = 0$, assign the following...

$$-Q_{0,j} = 2y_j^3 - 3y_j^2 + 1, -Q_{N-1,j} = 0 \quad \forall j$$

$$a_{0,j}^P = a_{N-1,j}^P = 1 \quad \forall j$$

$$a_{0,j}^W = a_{0,j}^E = a_{0,j}^S = a_{0,j}^N = a_{N-1,j}^W = a_{N-1,j}^E = a_{N-1,j}^S = a_{N-1,j}^N = 0 \quad \forall j$$

For the Neumann B.C. $T_{i,0} - T_{i,1} = 0, T_{i,N-1} - T_{i,N-2} = 0...$

$$a_{i,0}^P = 1, a_{i,0}^S = -1, a_{i,0}^W = a_{i,0}^E = -Q_{i,0} = 0 \quad \forall i \neq 0, N-1$$

$$a_{i,N-1}^P = 1, a_{i,N-1}^N = -1, a_{i,N-1}^W = a_{i,N-1}^E = -Q_{i,N-1} = 0 \quad \forall i \neq 0, N-1$$

Problem 1b

Jacobi method:

$$T_{i,j}^{k+1} = -\frac{Q_{i,j}}{a_{i,j}^P} - \frac{a_{i,j}^W}{a_{i,j}^P} T_{i-1,j}^k - \frac{a_{i,j}^E}{a_{i,j}^P} T_{i+1,j}^k - \frac{a_{i,j}^S}{a_{i,j}^P} T_{i,j-1}^k - \frac{a_{i,j}^N}{a_{i,j}^P} T_{i,j+1}^k$$

Gauss-Siedel method (from the bottom-left corner to the top-right corner of the domain):

$$T_{i,j}^{k+1} = -\frac{Q_{i,j}}{a_{i,j}^P} - \frac{a_{i,j}^W}{a_{i,j}^P} T_{i-1,j}^{k+1} - \frac{a_{i,j}^E}{a_{i,j}^P} T_{i+1,j}^k - \frac{a_{i,j}^S}{a_{i,j}^P} T_{i,j-1}^{k+1} - \frac{a_{i,j}^N}{a_{i,j}^P} T_{i,j+1}^k$$

Successive Overrelaxation (SOR):

$$T_{i,j}^{k+1} = T_{i,j}^k - \alpha \left[\frac{Q_{i,j}}{a_{i,j}^P} + \frac{a_{i,j}^W}{a_{i,j}^P} T_{i-1,j}^{k+1} + \frac{a_{i,j}^E}{a_{i,j}^P} T_{i+1,j}^k + \frac{a_{i,j}^S}{a_{i,j}^P} T_{i,j-1}^{k+1} + \frac{a_{i,j}^N}{a_{i,j}^P} T_{i,j+1}^k \right]$$

Problem 1c

The Jacobi method was ran until...

$$\|\{Q\} - [A]\{T\}^k\|_{L^2} < 10^{-2}$$

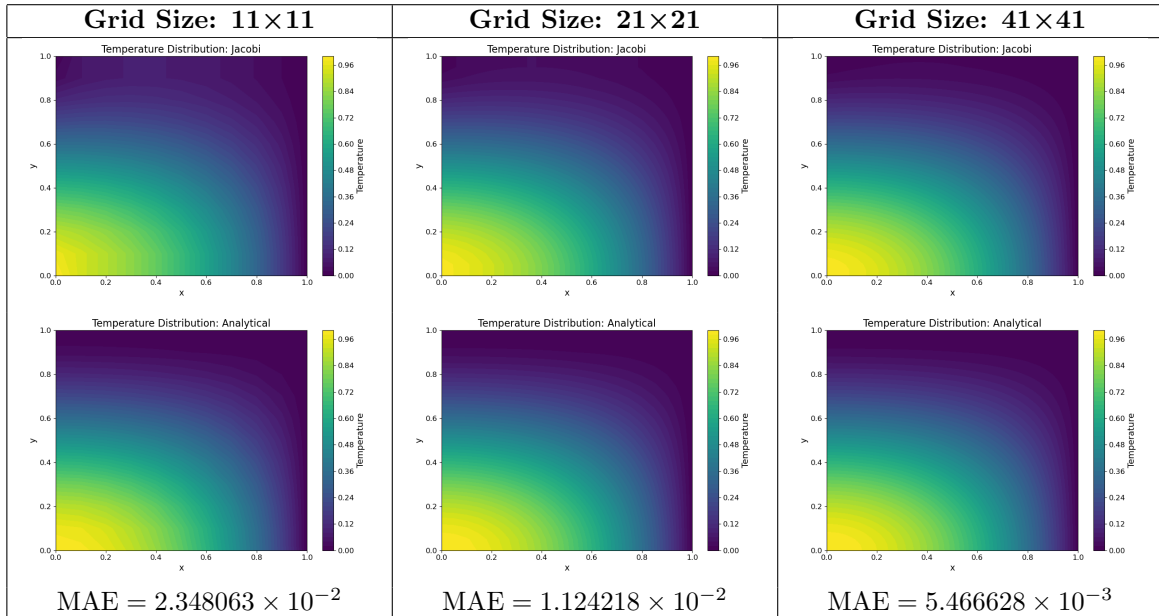
...using the initial condition...

$$T_{i,j}^0 = 2y_j^3 - 3y_j^2 + 1$$

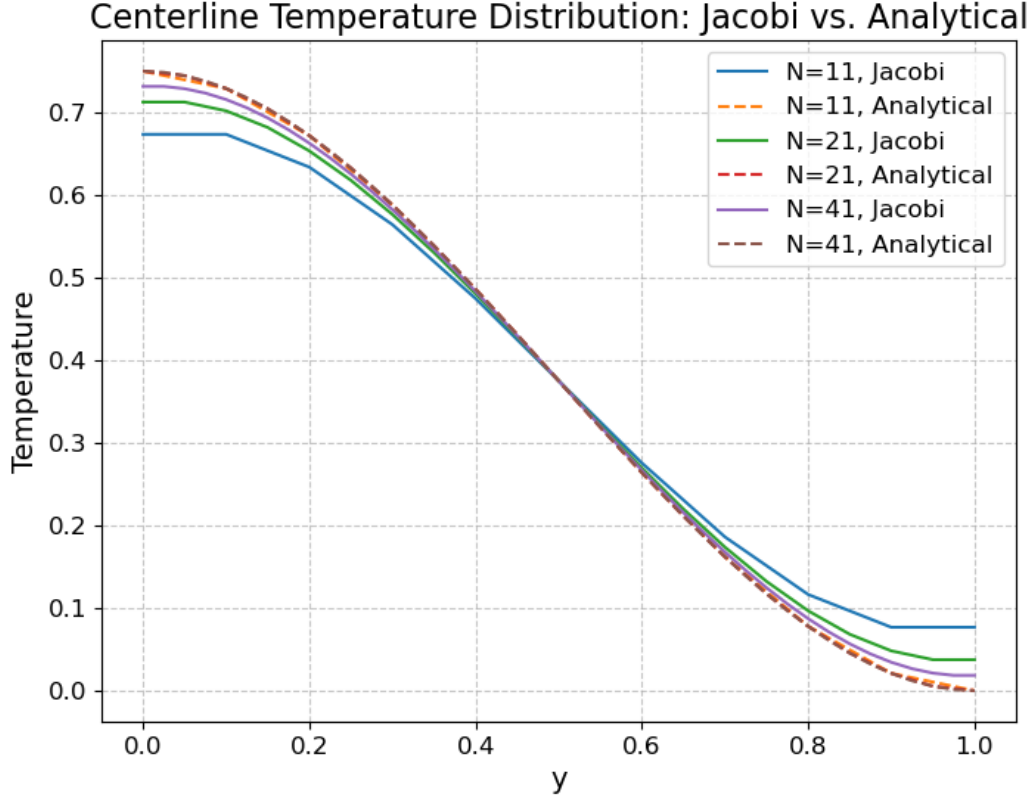
The mean absolute error (MAE) between the numerical and analytical was computed as...

$$\text{MAE} = \frac{1}{N^2} \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} |T_{i,j}^{\text{numerical}} - T_{i,j}^{\text{analytical}}|$$

Run `homework2c.sh` to regenerate all the graphs used.



To plot the temperature along the vertical centerline, the x-coordinate of the centerline in the finite difference grid was found by taking the floor of $\frac{N}{2}$.



Problem 1d

The spatial convergence rate of the finite difference scheme is shown to be roughly first-order with respect to h . The flux boundary condition is discretized as...

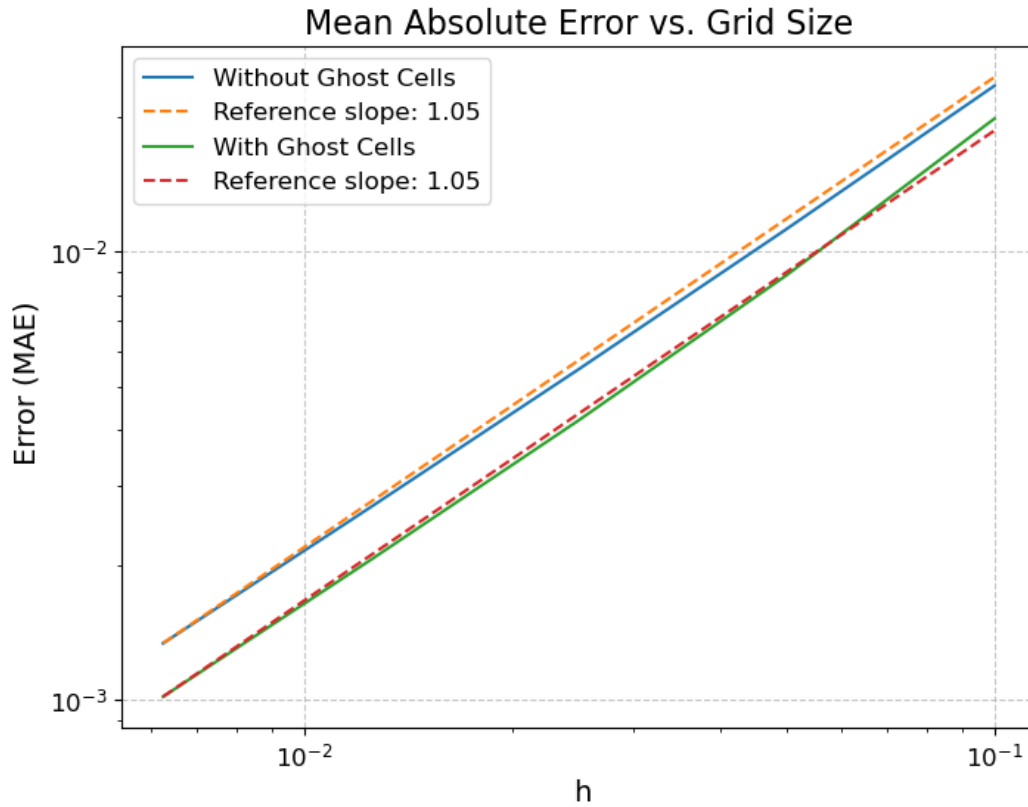
$$\frac{T_{i,0} - T_{i,1}}{h} = 0, \quad \frac{T_{i,N-1} - T_{i,N-2}}{h} = 0, \quad \forall i$$

This can be improved, however, by using ghost rows across the top and bottom boundaries of the domain and changing the discretization of the flux boundary condition.

$$\frac{T_{i,-1} - T_{i,1}}{2h} = 0, \quad \frac{T_{i,N} - T_{i,N-2}}{2h} = 0, \quad \forall i$$

Here, $j = -1$ and $j = N$ represents the ghost rows above and below the domain. From the graph below, there is a decrease in MAE when the flux boundary condition is discretized using ghost cells.

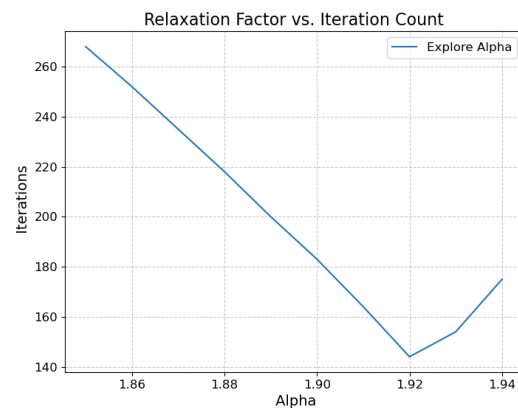
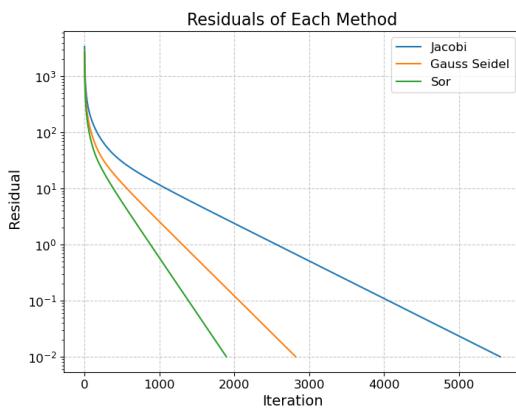
Run `homework2d.sh` to regenerate the graph below.



Problem 1e

For all three residual plots, $N = 41$ was used. For SOR, $\alpha = 1.92$ based on which relaxation factor resulted in the least number of iterations.

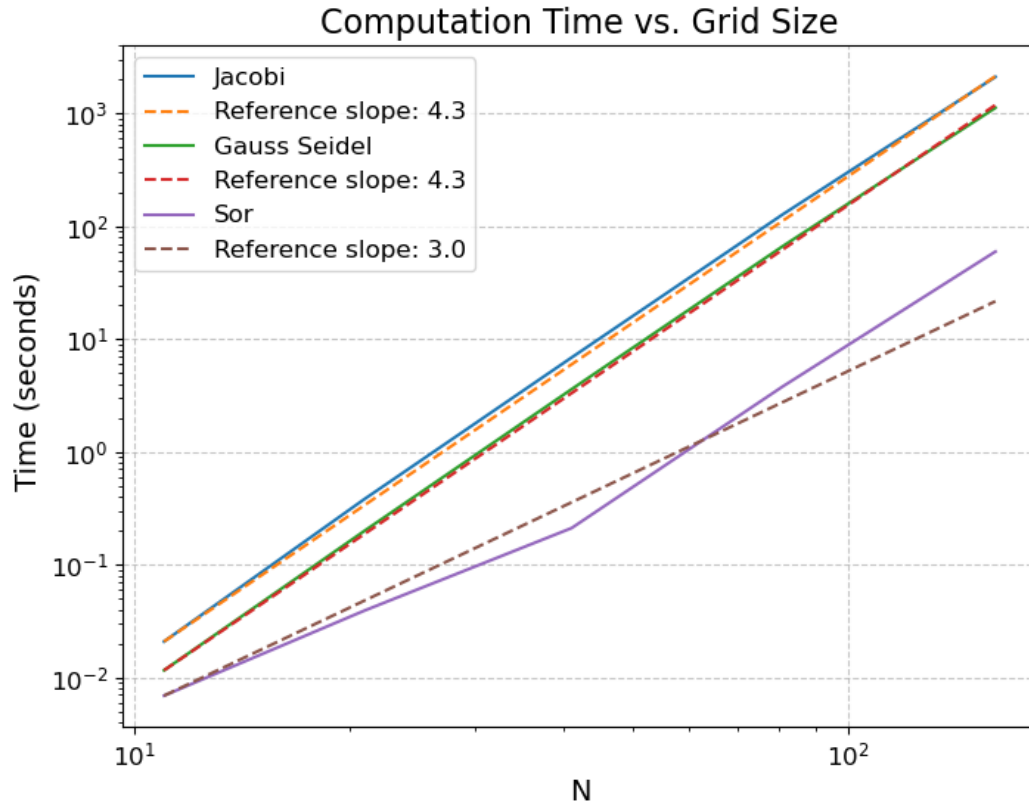
Run `homework2ei.sh` to regenerate the graph on the left. Run `homework2eii.sh` to regenerate the graph on the right.



Problem 1f

The three methods were run for $N = 11, 21, 41, 81, 161$ and their computation times are plotted using a logarithmic scale. From the graph, Jacobi and Gauss-Seidel have a time complexity of roughly fourth-order, whereas SOR is roughly third-order.

Run homework2fg.sh to regenerate the graph.



Problem 1g

A two-level multi-grid method was also developed and tested against the other three methods. With a looser tolerance of $\rho^k < 5 \times 10^{-2}$, the multi-grid method has a time complexity similar to that of the Jacobi and Gauss-Seidel methods.

Run homework2fg.sh to regenerate the graph.

